Strings

Strings are probably the most commonly used data type in Ruby, and they have a powerful and wide-ranging API to prove it. Here are some of the most useful and most interesting String methods.

Finding Information about a String

The length of a string is accessible with the methods length or size (the two methods are aliases). The length is in characters as determined by the current encoding. You can get the length in bytes with the method bytesize. The method empty? returns true if the length of the string is zero.

If you want to know how many times a given character is used in a string, you can use count. This works as you probably expect if you pass in a single character, but gets a little confusing if you pass in a longer string or multiple strings. For example:

```
"Banana bread".count("b")        # => 1
"Banana bread".count("ba")       # => 5
"Banana bread".count("bad", "a") # => 4
```

The argument to count is called a *character selector*. A multi-character string selector matches for any of the characters so included, so ba as an argument finds the counts for all characters that are b or a. A character selector can use a hyphen to suggest a range of characters, and a caret to invert the selection. For example:

```
"Banana bread".count("a-d") # => 6
"Banana bread".count("^ba") # => 7
```

The count method can take more than one of these selectors as arguments, in which case a character needs to be a part of *all* the selectors in order to be counted. In our "Banana bread".count("bad", "a") example, bad and a only overlap with a, so we get the count of a.

Testing the Content of Strings

The most generic way to determine if a string contains particular content is with the index method. The index method takes an argument that is either a string or a regular expression and returns the index of the first position in the receiver that matches the argument, returning nil if there is no match:

```
"The pickaxe book".index("ck") # => 6
"The pickaxe book".index(/\s/) # => 3
"The pickaxe book".index("z")  # => nil
```

An optional second argument is an index. If the index is a positive integer, it returns the first match after that index. If the index is a negative integer, index still returns the first match after the index, but it counts the index from the end of the string:

```
"The pickaxe book".index("e")     # => 2
"The pickaxe book".index("e", 4)  # => 10
"The pickaxe book".index("e", -7) # => 10
```

In the last line, the search starts at index -7, which is seven characters from the end of the string, but then moves to the end of the string and returns the index of the second e.